

It Is Not the Signature: A Discarded Key-Parse Attempt Dominates Token Validation Cost in a Node.js Service

Jannatul Ferdous Katha^{1,3}, Tasmia Tahmid Prova^{2,3},
Md Kishor Morol³, Tze Hui Liew^{4*}, Dip Nandi^{3,5}

¹Department of CSE, Bangladesh University of Professionals (BUP),
Dhaka, Bangladesh.

²Department of CSE, Bangladesh University of Business and Technology
(BUBT), Dhaka, Bangladesh.

³ELITE Research Lab, Queens, NY, USA.

⁴Faculty of Information Science and Technology, Multimedia University,
Melaka, Malaysia.

⁵Department of Computer Science, American International
University-Bangladesh (AIUB), Dhaka, Bangladesh.

*Corresponding author(s). E-mail(s): thliew@mmu.edu.my;

Contributing authors: ferdous.katha@elitelab.ai; tasmia.prova@elitelab.ai;
kishormorol@ieee.org; dip.nandi@aiub.edu;

Abstract

A performance cost is only as well attributed as the components recorded alongside it. We show this for JSON Web Token validation, whose per-request cost is routinely attributed to cryptography. In `jsonwebtoken`, the widely used Node.js library, the signature primitive is 2.208 μ s of a 29.92 μ s call. The dominant component, 18.04 μ s or 60%, is a discarded parse: given a shared secret as a string, the library first tries to parse it as an asymmetric key and falls back only after that attempt fails. Differencing across 15 processes and a sampling profiler agree on this. Its price is set by the OpenSSL release a runtime bundles, and the evidence is causal. The same OpenSSL calls issued from C, against 8 identically built releases, cost 394.4 μ s on 3.0.16 and 8.447 μ s on 3.5.8. A newer JavaScript engine paired with an older OpenSSL is the slow runtime, and the exception V8 builds costs a few microseconds. In deployment, validation adds 384.6 μ s of processor

time per request, within 0.8% of the microbenchmark prediction. Of seven JWT libraries in four languages, only this one attempts the parse; five pay nothing for a raw secret. The avoiding pattern is rare in public code, and the obvious fix would disable a key-type defence that the discarded parse makes possible; we give a narrow fix that preserves it and have reported the defect upstream.

Keywords: JSON Web Token, JWT, key resolution, performance measurement, benchmarking methodology, Node.js, OpenSSL 3.x, software defect

1 Introduction

Authentication in cloud-native services is commonly described as imposing a per-request cryptographic cost (Zhu et al, 2023). The assumption is old and has been tested before, one layer down: Coarfa et al (2006) instrumented TLS web servers and found RSA responsible for only a minority of server cost, the remainder falling to symmetric operations, hashing and protocol handling. Our result is that argument repeated at the application layer, and reaching a sharper version of the same conclusion.

The reasoning being tested is intuitive: validating a JSON Web Token (Jones et al, 2015) means verifying a signature, signatures are cryptography, and cryptography is expensive relative to parsing. The conclusion usually drawn is that the cost should be *moved* — to a sidecar proxy, a gateway, or dedicated hardware — because specialised components do cryptographic work well.

We report that the premise is wrong for the most widely used JavaScript implementation we could measure. The natural repair is wrong as well. Once the signature primitive is shown to be a small share of the cost, the remainder is readily attributed to key material being converted from a string into a key object on every call. Measuring that conversion directly puts it at 0.583 μ s: too small to be the explanation for anything.

The actual mechanism is a control-flow accident, shown in Figure 1. Given a secret that is not already a key object, the library attempts `createPublicKey()` first and falls back to `createSecretKey()` in the exception handler. For an HMAC secret the first call cannot succeed. Every validation therefore constructs and discards an OpenSSL parse failure, at 18.04 μ s against a total call cost of 29.92 μ s.

We are careful about what that 18.04 μ s is. It is the cost of the whole discarded attempt: the work OpenSSL performs before deciding the input is not an asymmetric key, plus the construction and unwinding of the exception that carries that decision back to the library. ~~We measure the pair. We do not separate them, and~~ Section 4.2 states the bound our data does place on the split. ~~The paper’s title and claims are pitched at the granularity we measured~~ separates the two, by reissuing the OpenSSL calls from C and by measuring an exception of the same shape with no OpenSSL call. Where the attempt is expensive it is almost all parse; the exception is a few microseconds. The title’s “key-parse attempt” is therefore the accurate name, and “discarded exception” would not be.

Research questions

RQ1 Where does the per-request cost of token validation actually go?

RQ2 What ~~does-governs~~ the magnitude of that cost ~~co-vary-with~~?

RQ3 Does it matter in a deployed service, and how common is the pattern that avoids it?

RQ4 Is the defect a property of one library, or of a design choice other libraries could share?

Contributions

All supported by measurements in the replication package (Katha et al, 2026).

1. A decomposition of the validation call, by two independent methods, locating its dominant component in a discarded key-parse attempt rather than in the signature primitive or in key conversion, and a second decomposition of that attempt into the OpenSSL parse, the V8 exception and the binding between them (Section 4).
2. ~~Evidence that the magnitude co-varies with the runtime's bundled OpenSSL version rather than with containerisation, together with an explicit statement of what the available runtime pairs cannot separate~~ A causal attribution of its magnitude to the bundled OpenSSL release: the parse reissued from C with no JavaScript engine across 8 identically built OpenSSL releases, a runtime pair that crosses engine and library, and a repetition on a second architecture (Section 5).
3. A deployed-service measurement across seven arrival rates, and a survey establishing that the avoiding pattern is rare in three samples of public code (Section 6).
4. ~~A fix, and~~ comparison of seven JWT libraries in four languages showing that the cost follows a design choice — resolving key material on every call, of which a failing asymmetric parse used as a type test is the costliest form —

rather than JWT validation as such (Section 7).

5. A fix, a constraint on it, and an upstream report: the discarded attempt is load-bearing for an existing key-type defence, so the natural form of the optimisation is unsafe (Section 8).

2 Background and related work

What a validation call does.

A JSON Web Token carries a header, a payload and a signature over the first two (Jones et al, 2015). Validating one means decoding two segments, resolving key material into a form the cryptographic library accepts, and verifying the signature. The middle step — key resolution — is the subject of this paper, and it is the step most often left implicit when validation cost is discussed.

Where the cost is usually attributed.

Published measurement of layered enforcement in microservices concentrates on transport and proxy layers. Zhu et al. Zhu et al (2023) decompose service-mesh sidecar overhead and show that the dominant contributors depend on configuration and workload rather than being a fixed tax; we reach an analogous conclusion one layer up, for application-level validation. Application-level token validation is itself thinly evidenced, and is commonly reported as a difference between scenario averages, which attributes to validation every difference between two runs. We were unable to locate a study that measures the key-resolution step directly; we state that as an absence we searched for rather than as a claim about the whole literature.

Cryptographic cost has been over-attributed before.

~~Coarfa et al.~~ Coarfa et al (2006) remains the clearest precedent: measuring TLS web servers rather than primitives, they found the public-key operation was a minority of server cost. Work on cryptographic library performance since has largely stayed at the level of the primitive or the protocol exchange, ~~with Tuveri and Brumley~~ (Naser et al, 2020; Sikeridis et al, 2020), ~~with Tuveri and Brumley~~ (2019) a notable exception in examining how a library loads and dispatches implementations rather than how fast those implementations run. Our result concerns neither: it is the cost of getting key material *into* the library.

Performance bugs as a class.

Jin et al (2012) studied real-world performance bugs across five code bases and found that most follow a small number of recurring patterns — among them uncoordinated and skippable function calls — and that fixes are usually local. Zaman et al (2012) found, from 400 Firefox and Chrome reports, that performance bugs differ from other bugs in how they are reported, discussed and fixed, in part because reproducing them is hard; Nistor et al (2013a) report that most are found by code reasoning rather than by observing symptoms. Liu et al (2014) reach similar conclusions for mobile applications. Detection work targets structural symptoms: redundant traversals with similar memory-access patterns (Nistor et al, 2013b), or predicates that separate slow from fast runs (Song and Lu, 2014). The defect here is of the “skippable call” kind Jin et al. describe, but it has none of the structural signatures those detectors look for: it is one call, executed once

per request, whose failure is its only observable effect. It is also the shape that makes a performance bug hard to report: our own upstream report was preceded by a two-year-old report of the symptom with no diagnosed cause (Section 8).

JavaScript and Node.js performance.

Selakovic and Pradel (2016) classified performance issues in JavaScript projects and found that inefficient API usage — correct calls made in a costly way — is among the most common root causes, and that the fixes are typically a few lines. Their later profiler (Selakovic et al, 2017) targets precisely the order in which a program evaluates conditions, the axis on which this defect lives: a check that almost always fails is evaluated before one that almost always succeeds. Gong et al (2015) locate code that defeats JIT optimisation: exception-heavy control flow is one such pattern, and Section 4.2 measures what it costs here. Work on the Node.js event loop shows why a per-request synchronous cost matters beyond its magnitude: a single slow handler delays every other client (Davis et al, 2018; Staicu and Pradel, 2018). We make no availability or capacity claim, but the mechanism by which a synchronous per-request cost propagates is the same.

Version and environment variation.

That software performance changes across versions in ways maintainers do not anticipate is established: ~~Kaltenecker et al.~~ Kaltenecker et al (2023) track performance across configurable systems’ release histories, and ~~Chen and Shang~~ Chen and Shang (2017) study the code changes that introduce regressions. ~~Laaber and Leitner~~ Laaber and Leitner (2018) assess

whether microbenchmark suites detect such changes at all, and Stefan et al (2017) find performance unit testing rare in practice. Container images make this concrete by pinning a dependency set implicitly: ~~Cito et al.~~ Cito et al (2017) characterise how the ecosystem uses them, and ~~Malka et al.~~ Zerouali et al (2019) show that images routinely ship outdated packages, and Malka et al (2026) show that an image tag does not guarantee a reproducible build. For OpenSSL 3 specifically, the maintainers of a major downstream library have stated publicly that parsing and key loading regressed substantially against 1.1.1 (Gaynor and Kehrer, 2026); that statement is not peer reviewed and was not produced under a protocol we can assess, and Section 5.5 supplies a controlled measurement for the one path this paper concerns. Section 5 is an instance with a measured cost attached.

Measurement protocol is the thing at stake.

The methodological core of this paper is not new. ~~Georges et al.~~ Georges et al (2007) established that a single execution of a managed-runtime benchmark is not a measurement, and that intervals must be computed across executions rather than within one. ~~Mytkowicz et al.~~ Kalibera and Jones (2013) show how to spend a fixed budget across levels of repetition, which is the reasoning behind our choice of many short processes over few long ones. Mytkowicz et al (2009) showed that environmental details unrelated to the code under study can produce effects larger than the one being measured, without any obvious mistake. ~~Traini et al. found that steady state is frequently not reached at all and,~~ and Curtsinger and Berger (2013) that

memory layout alone can do so. Barrett et al (2017) found that virtual machines frequently do not reach a steady state, Traini et al (2023) that developers misjudge when warmup ends, and Costa et al (2021) catalogue how microbenchmarks go wrong in practice. Mytkowicz et al (2010) showed that sampling profilers disagree on which method is hot, which is why Section 4 uses the profiler only as corroboration of a result obtained by differencing and relies on ordering rather than on its percentages. Our contribution is not to these methods but is an instance of what they warn about. The attribution this paper overturns has exactly the structure Mytkowicz et al. describe: a second variable — the bundled OpenSSL version — moving unrecorded alongside the one under study, at a granularity coarse enough to hide it (Section 5).

Environment as a confounder.

~~Laaber et al.~~ Laaber et al (2019) quantify how far cloud environments degrade microbenchmark reliability, and ~~Leitner and Cito~~ Leitner and Cito (2016) characterise performance variation across public infrastructure providers. Their concern is variance between nominally identical environments. Ours is adjacent and, we think, less examined: a *systematic* difference between environments that are not identical and are routinely treated as though they were — two container base images differing in a bundled library version. Laaber et al. are also the reason the second-architecture repetition in Section 5.6 is reported as an ordering check rather than as a replication of magnitudes.

Cryptographic interfaces are hard to use correctly.

The defect reported here is not a cryptographic weakness but a consequence of how a cryptographic interface is invoked, which is a well-documented class. [Egele et al.](#) — [Lazar et al \(2014\)](#) found that most of the cryptographic vulnerabilities they studied were misuses of libraries by applications rather than bugs in the libraries. [Egele et al \(2013\)](#) found cryptographic misuse widespread in deployed applications and [Georgiev et al.](#) [Georgiev et al \(2012\)](#) found certificate validation broken across a wide range of non-browser software; [Nadi et al.](#) [Nadi et al \(2016\)](#) attribute much of it to the interfaces themselves rather than to developer inattention, a position [Green and Smith](#) [Green and Smith \(2016\)](#) put directly and [Indela et al.](#) [Indela et al \(2016\)](#) respond to with interface proposals; and [Aear et al.](#) [Acar et al \(2017\)](#) show experimentally that API design materially changes whether developers produce correct code. [Detection has been automated by specifying correct usage, as CrySL does \(Krüger et al, 2018\), and by static analysis at scale \(Rahaman et al, 2019\).](#) Our finding sits within that line with one difference: the interface here admits a use that is *correct*, and costly only for a reason invisible at the call site. [A CrySL-style rule could express “pass a key object, not a string”, but the rule sets we know of encode security properties only, so none would report it.](#)

JWT and JOSE libraries.

[Security analyses of JWT implementations consistently find key-handling defects. Detering et al \(2017\) demonstrated attacks, key confusion among them, against several popular JOSE libraries; Xu et al \(2023\) automated the detection of cryptographic](#)

[vulnerabilities in applications that use JWT; Yang et al \(2026\) evaluated 43 JWT libraries across 16 programming languages for vulnerabilities; and Shatnawi et al \(2024\) surveyed Python JWT libraries for security warnings and adoption. RFC 8725 \(Sheffer et al, 2020\) codifies the resulting practice. All of this work treats a library’s handling of key material as a security surface. Section 7 treats the same surface as a cost site, across seven libraries, and Section 8 shows the two views are coupled: the control flow that costs time in one library is the control flow that enforces its key-type check.](#)

Mining public repositories.

Section 6.5 draws on code hosted publicly, and the hazards of doing so are catalogued by [Kalliamvakou et al.](#) [Kalliamvakou et al \(2014\)](#) — among them that many repositories are not software projects, that activity is not uniformly distributed, and that a search result is not a random sample. Our sampling frame is stated in Section 3.9 with those hazards in view, which is why no population proportion is computed from it.

The cost of exceptions.

That exceptions are expensive in managed runtimes, and that their cost is concentrated in construction rather than in unwinding, is long established: [Ogasawara et al.](#) [Ogasawara et al \(2006\)](#) optimise around exception paths in a Java virtual machine, and [Renwick et al.](#) [Renwick et al \(2019\)](#) quantify the cost in a compiled setting. [Parravicini and Mueller](#) [Parravicini and Mueller \(2021\)](#) provide the closest available cost-attribution baseline for the V8 engine, though on speculation rather than exceptions. ~~None measures what our result turns on;~~

~~and Section 4.2 states the consequence: we report the parse attempt and its terminating exception as one quantity because we have no instrument that separates them.~~ Empirical studies of how exceptions are *used* find handlers that discard what they catch to be common: Cabral and Marques (2007) report that handlers in the Java and .NET code they studied rarely attempt recovery, Kery et al (2016) find that handlers mostly log, print, return or rethrow, and de Pádua and Shang (2017) measure the prevalence of handling anti-patterns, as Weimer and Nacula (2004) did earlier for error-handling mistakes. That literature treats a swallowed exception as a correctness and maintainability smell. The handler at the centre of this paper is a swallowed exception used as control flow on a hot path, and it is a performance defect rather than a correctness one: the fallback it guards is correct. None of these studies measures such a handler’s cost; Section 4.2 does, and finds the exception itself the smaller part.

Key-type confusion.

That a token declaring an HMAC algorithm must not be validated against asymmetric key material is long established; RFC 8725 (Sheffer et al, 2020) describes the attack and its mitigations. Section 8 reports no new attack. It reports that an existing defence depends on the control flow whose cost this paper measures, so that removing the cost naively removes the defence.

2.1 What we searched for and did not find

~~Five-Three~~ areas bear directly on this result and we were unable to locate verifiable peer-reviewed work in any of them.

We state each as an absence we searched for rather than as a claim about the whole literature; each is also, in part, why the misattribution corrected here survived as long as it did. Two further gaps recorded in an earlier draft — error-construction cost in V8, and attribution of key-parsing cost inside OpenSSL — are now closed by measurements in this paper (Sections 4.2 and 5.5) and are no longer listed.

OpenSSL 3.x performance regressions. ~~Section 5 reports a magnitude that co-varies with a bundled OpenSSL version.~~ We found no peer-reviewed study of performance regressions across the OpenSSL 3.x series. Evidence exists ~~but as a maintainer statement~~ (Gaynor and Kehr, 2026), as project issues ~~and mailing-list discussion~~ — for instance openssl/openssl issues #17627 and #20698, which concern performance of the 3.x provider and decoder machinery ~~. We cite these as engineering evidence and note explicitly that they are not peer-reviewed and were not — and as mailing-list discussion.~~ None was produced under a measurement protocol we can assess. ~~Consequently Section 5 reports the co-variation we measured and does not adopt, so Section 5.5 measures the path directly rather than adopting the mechanism those issues sources propose.~~

Base image to bundled library to application performance. We found no academic study tracing that chain. ~~Malka et al.~~ Malka et al (2026) address build reproducibility ~~and Cito et al.~~, Zerouali et al (2019) package staleness and Cito et al (2017) ecosystem practice; ~~neither none~~ measures how the library version an image pins changes application performance.

Cryptographic API misuse as a performance defect. The misuse literature cited

above treats misuse exclusively as a security problem. We found no work treating a correct-but-costly use of a cryptographic interface as a latency or throughput defect, which is precisely what this paper reports.

~~Error construction cost in V8 or Node.js. Our result turns on the cost of a discarded parse attempt whose terminating exception is a plausible component. We found no measurement of Error construction cost specific to V8 or Node.js, so we have no prior baseline to compare against and do not decompose that cost here. Section 4.2 states what follows.~~

~~Profiling inside a cryptographic library. We found no study that profiles within a cryptographic library to attribute time to key parsing and loading rather than to the primitive. Section 4 does this from outside the library, by differencing and by frame attribution, which is weaker than instrumenting it directly and is the reason Section 5's attribution is stated as a co-variation.~~

3 Methodology

3.1 Unit of measurement

The unit is one operating-system process, not one call. Each process performs 40000 timed iterations of a single condition after a warmup of half that number, and contributes its median. Reported figures are medians of per-process medians, with percentile bootstrap intervals taken over processes.

An interval computed over the calls inside one process describes that process's steady state and is far too narrow: it cannot observe the compilation decisions, inline-cache states and heap layouts that differ between runs of the same code (Georges et al, 2007). Conditions in the decomposition of Section 4 were run in

15 separate processes of 40000 calls; conditions in the runtime matrix of Section 5 in 10 processes of 20000 calls. The largest between-process coefficient of variation is 3.5% on the host; in the matrix it is 20.2%, occurring on the probe_succeeds condition under node:26.6.0-bookworm rather than across the matrix generally.

Bootstrap intervals over 15 or 10 units are coarse, and we report them as indicative rather than as calibrated coverage. Where an interval collapses to a point — the timer overhead row of Table 1 — it should be read as the clock's resolution showing through, not as a precise estimate. The per-process values underlying every interval are in the replication package.

Conditions are interleaved within each round rather than run in blocks. Under block ordering a thermal excursion or a background process lands entirely on whichever condition is executing and is indistinguishable from an effect; interleaved, it is distributed across conditions and appears as between-round variance. The clock's own cost is measured in the same process as the condition it accompanies and is reported rather than subtracted: 0.042 μ s against the smallest quantity reported, 0.583 μ s.

Differences between conditions are reported throughout as the median of per-round *paired* differences, not as the difference of two medians. The two are not identical and the distinction matters at the magnitudes involved; Section 4.3 and Section 8 each rely on it.

3.2 Decomposing the call

`jwt.verify()` is decomposed into the operations it performs, each timed in isolation under the same protocol: structural decode; the HMAC primitive with a string key and with a pre-parsed key; the


```

120 if (secretOrPublicKey != null && !(
    secretOrPublicKey instanceof
    KeyObject)) {
121   try {
122     secretOrPublicKey =
      createPublicKey(secretOrPublicKey
    );
123   } catch (_) {
124     try {
125       secretOrPublicKey =
        createSecretKey(typeof
        secretOrPublicKey === 'string' ?
        Buffer.from(secretOrPublicKey) :
        secretOrPublicKey);
126     } catch (_) {
127       return done(new
        JsonWebTokenError('
        secretOrPublicKey is not valid
        key material'))
128     }
129   }
130 }

```

Fig. 1 Key resolution in `jsonwebtoken` version 9.0.3, `verify.js`, lines 120--130. Verbatim but for the common leading indentation, which is removed to fit the column; line numbers are the file's own. A secret that is not already a `KeyObject` is offered to `createPublicKey()` first, and `createSecretKey()` is reached only from the exception handler. For an HMAC secret supplied as a string the outer `try` can never succeed, so the asymmetric parse is attempted and discarded on every validation. The excerpt is extracted from the pinned release by `analysis/make_codepath_figure.py` and pinned by content hash `sha256:4c4781826f49fa90` in the replication package (Katha et al, 2026).

symmetric key constructor; and the asymmetric key parse the library attempts first, in both the case where it throws and the case where it succeeds.

The library measured throughout is `jsonwebtoken` version 9.0.3, cloned at tag v9.0.3. The code path at issue is shown in Figure 1.

3.3 Separating the runtime from the deployment

A host microbenchmark compared against an in-container measurement cannot support an attribution to deployment: those

two environments differ in more than deployment, and any of the differences could carry the effect. Here containerisation is held constant and the runtime is varied instead: four container images spanning two OpenSSL series and three V8 major versions, on one machine under one container engine.

The pair carrying most of the inference is Node 18 and Node 20: they ship V8 10 and V8 11 respectively while sharing the OpenSSL 3.0 series. In stock images Node and OpenSSL versions otherwise covary, which would leave the attribution underdetermined. Because that pairing carries the attribution, the runtime identity of each image — Node, OpenSSL and V8 version, libc, architecture and image digest — is recorded as a datum and cross-checked against the versions each measurement row carried. That check found 0 mismatches across the matrix. Section 5.2 states what this design ~~still cannot separate alone cannot separate,~~ and Sections 3.4 and 3.5 describe the two further designs that do.

3.4 Taking the JavaScript engine out

Runtime pairs can separate engine from library only as far as stock builds allow, so the attribution was tested by removing the engine altogether. We read the Node source at the two release lines measured (`src/crypto/crypto_keys.cc` at v18.20.8; `deps/ncrypto` at v22.23.3, which issues the same sequence) and reproduced in C. call for call, what `createPublicKey()` asks of OpenSSL when handed an HMAC secret: three `PEM_bytes_read_bio` scans for a public key, an RSA public key and a certificate, each under an error-queue mark and each followed by `BIO_reset`; then, since none was recognised,

`PEM_read_bio_PrivateKey` with Node's passphrase callback; then the C side of Node's error path, which drains the OpenSSL error queue into strings. The V8 side of that error path is deliberately absent; it is what Section 3.5 measures.

The program was compiled once per OpenSSL release with one compiler (GCC 13.3.0) and one set of flags, against 8 releases from 3.0.16 to 3.6.1, each built from source with identical configuration and linked by run path so that no system library could be picked up. Each binary reports the version it actually loaded; rows whose loaded version differed from the one requested would be discarded, and there were 0. The protocol is that of Section 3.1: one condition per process, 15 processes per release and condition, 5000 timed calls each, timed with `clock_gettime`, conditions and releases interleaved within rounds. Besides the full sequence we time its parts separately — the three public-key scans, the private-key parse, the error-string drain — and a *successful* parse of an RSA-2048 public key through the same path.

3.5 Separating the exception from the parse

Inside Node the failed parse is an OpenSSL failure followed by a V8 exception: Node constructs an `Error`, attaches `library`, `reason` and `code` properties, throws it through the `createPublicKey()` frame, and the caller discards it. To measure that exception without the parse, a JavaScript benchmark builds the same object — message, property values and error code are captured from a real failure on the runtime being measured, not typed — and throws it from 8 frames below the catch. Variants separate its parts: a plain

`Error` thrown and caught in one frame; the Node-shaped error at depth; the same with stack capture disabled; and a Node-internal argument error raised before any OpenSSL call. The real probe is timed alongside, with and without stack capture, so that the stack's share of it is obtained by paired differencing. All conditions run under official Node release binaries, which bundle their own OpenSSL, on six runtimes — 18.20.8, 20.20.2, 22.23.3, 23.11.1, 24.21.0 and 26.10.0 — at 15 processes of 10000 calls per condition.

Combining this with Section 3.4 gives a three-way decomposition of the probe on each runtime: the C cost of the OpenSSL release that runtime bundles, the V8 exception, and a remainder we attribute to the binding between them — argument preparation, key-handle allocation and the C++ side of error construction. The three come from different processes and cannot be paired, so the interval on the remainder is obtained by resampling each run's processes independently.

3.6 Other libraries

To ask whether the defect is one library's or a design choice's, we read the key-handling code of six further libraries, chosen as the most used in each ecosystem: `fast-jwt` and `jose` (JavaScript), `PyJWT` (Python), `golang-jwt` (Go), and `nimbus-jose-jwt` and `jjwt` (Java). For each we answer one question from source: given a string HMAC secret, does it attempt to parse it as an asymmetric key? We then time HS256 verification in each with the secret in two forms, under the same one-condition-per-process protocol at 15 processes of 20000 calls. "String" is the form each library's documentation

shows for an HMAC secret, used the way its interface is meant to be used: “pre-parsed” is the library’s own key object, built once. Where a library does not accept a string we use its rawest accepted form and say so. The JavaScript libraries are run under two runtimes, one bundling OpenSSL 3.0 and one bundling 3.5, because the `jsonwebtoken` penalty depends on that. The penalty is the median of per-round paired differences between the two forms.

3.7 A second architecture

Tables 1 and 3 were measured on one ARM64 host. Both were repeated with unchanged scripts on an x86_64 machine (Intel(R) Xeon(R) Processor @ 2.80GHz, four virtual CPUs), including the four container images of Table 3; Sections 3.4–3.6 were measured on the same machine. That machine is a virtual machine on shared cloud infrastructure, not the dedicated host a replication of magnitudes would require (Laaber et al, 2019; Leitner and Cito, 2016). We use it for what it can support: whether the ordering of conditions and runtimes holds on a second instruction set, and ratios between conditions measured interleaved in the same rounds. We do not compare its absolute magnitudes with the ARM64 host’s.

3.8 In-situ measurement

The deployed service exposes a histogram timing the verification call alone. The instrument is a *paired-scrape difference*: the cumulative histogram is read before and after each arrival-rate step, and the two readings are differenced, which isolates that step’s distribution from the cumulative counters. Means are computed as the difference of the histogram’s `_sum`

divided by the difference of its `_count`, so they are exact and do not depend on bucket boundaries; bucket resolution bounds the reported percentiles only.

Seven arrival rates are driven, and the whole sweep is run twice on one process: once in ascending order and once in descending order. We report the *range across the two passes*. This is a range of two, not a confidence interval; no standard error is computed from it, and it is not written as a symmetric tolerance. The opposite-order design also serves as a drift test, which Section 6.2 reports.

One process and two passes is the whole of the in-situ evidence. It is sufficient to establish that the cost is present in deployment and of what order; it is not sufficient to characterise the shape of the cost-versus-rate relation, and Section 6.1 is written accordingly.

3.9 Prevalence: three instruments, three frames

Prevalence is approached with three instruments that fail in different ways. Their sampling frames differ and are not interchangeable, so no population proportion is computed from them. All three were captured on 2026-09-18.

Population co-occurrence counts. Code-search counts over public repositories, in four cells: two languages by two import forms. The denominator of a cell is files importing the library; the numerator is files that *also mention* the pre-parsing interface. This measures co-occurrence of two strings in one file, not verified usage. The denominators are returned rounded — each is an exact multiple of 1024 — so they are magnitudes rather than counts established file by file.

A classified sample of call sites. Call sites located by search, deduplicated to one file per repository so that no project

dominates, with the key argument traced within its file. This is a convenience sample from a relevance-ranked, undocumented ranking; inclusion probabilities are unknown and no weighting back to the population is possible.

A targeted counter-search. Every file co-occurring the library with the pre-parsing interface, fetched in full and adjudicated by hand. This frame is deliberately biased toward finding the pattern — that is its purpose as a negative control — and it is therefore disqualified as a prevalence estimator.

Three reasons preclude a population-level prevalence. The population numerator and the verified numerator measure different things: of 13 automatically flagged call sites, 5 were false positives on hand inspection, so the co-occurrence signal carries an error rate that would be inherited, unmeasured, by any extrapolation. The sample has no inclusion probabilities. And the counter-search is selected on the outcome. Results are therefore reported *in our sample*, or *among repositories that co-occur the interface and call the validator*, and never as a population proportion. This constraint is carried into the abstract and the conclusion.

3.10 Host state is recorded per measurement

A service measured on a host it shares with its own load generator, ingress and sidecars is measured together with whatever else that host is doing. We observed this directly while building the instrument: re-running one configuration with a background indexing process consuming host CPU moved the point at which throughput collapsed by hundreds of requests per second, and both runs are committed in the replication package (Katha et al, 2026). A curve that moves with

unrelated host activity does not characterise the service, and this paper therefore makes no capacity claim. Every in-situ window instead carries the host load average observed during it, committed beside the measurement, and Section 6.3 uses it.

4 RQ1: Where the cost goes

4.1 The decomposition

Table 1 gives each operation in the validation path, timed in isolation on the host under the protocol of Section 3, and Figure 2 plots it.

The two emphasised rows are the result. The signature primitive costs 2.208 μ s of a 29.92 μ s call. The conversion to which the remainder had been attributed, including by us, costs 0.583 μ s. The discarded attempt that precedes that conversion costs 18.04 μ s, which is 60% of the whole call and its largest single component.

Supplying a pre-parsed key instead of a string reduces the call by 23.92 μ s [23.58, 24.00] over 15 paired rounds.

The natural repair is also wrong.

Once the signature primitive is shown to be small, one explanation for the remainder suggests itself immediately: that the cost lies in converting key material from a string into a key object on every call. It is the explanation we held before measuring it, and Table 1 refutes it. That conversion costs 0.583 μ s, so the attribution is wrong by roughly the ratio between that figure and 18.04 μ s.

A second observation is usually offered in support of the conversion account: an asymmetry between HMAC and RSA costs, read as conversion being dearer than

Table 1 Cost of each operation in the validation path. Medians of per-process medians over 15 processes of 40000 calls; 95% percentile bootstrap intervals over processes, which with 15 units are indicative rather than calibrated (Section 3.1).

Operation	Median (μs)	95% interval (μs)
verify() , string secret	29.92	[29.75, 30.12]
discarded asym. parse	18.04	[17.83, 18.25]
successful asym. parse	13.83	[13.62, 13.92]
verify() , pre-parsed key	6.042	[5.958, 6.125]
HMAC, string key	2.208	[2.208, 2.250]
HMAC, pre-parsed key	2.166	[2.125, 2.167]
structural decode	1.292	[1.250, 1.292]
symmetric key constructor	0.583	[0.542, 0.583]
timer overhead	0.042	[0.042, 0.042]

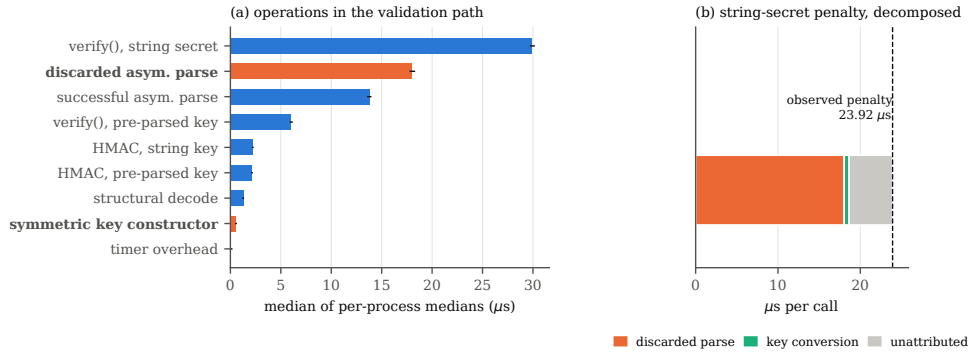


Fig. 2 (a) Every operation in Table 1, on a linear scale, with the two components the paper turns on emphasised; whiskers are the 95% percentile bootstrap intervals over 15 processes. (b) The 23.92 μs penalty for supplying a string secret rather than a pre-parsed key, decomposed. The three segments are medians of per-round paired differences, not a partition of a single measurement, and they do not sum to the penalty: the 5.292 μs remainder is unattributed (Section 4.3). Both panels are generated from `figures/data/` by `figures/make_figures.py` (Katha et al, 2026).

parsing a PEM. It is backwards. A successful asymmetric parse costs 13.83 μs against 0.583 μs for the conversion. The real asymmetry is that the RSA path’s parse *succeeds*, making its cost useful work, while the HMAC path’s fails, making it waste.

4.2 What the 18.04 μs is made of

The 18.04 μs is the cost of the discarded attempt taken whole. It contains at least two things: the work OpenSSL performs before concluding that the input is not an asymmetric key, and the construction and unwinding of the exception that returns that conclusion to the library. ~~We did not separate them. We state this prominently because the distinction bears directly on~~

the mechanism, and because a reader could otherwise take the headline as an attribution to exception machinery alone.

One bound is available from Table 1 and we state it as a bound rather than as a decomposition. An earlier version of this paper could only bound the split, from the observation that a *successful* asymmetric parse — the same entry point, reaching a conclusion, and throwing nothing — costs 13.83 μ s. The failing parse costs μ s. The two paths are not identical in the work they perform before diverging, so the, the same order as the failing one. Table 2 replaces the bound with a decomposition, combining the C measurement of the same OpenSSL release (Section 3.4) with the exception baseline (Section 3.5) on six official runtimes.

Three things follow. First, on every runtime bundling OpenSSL 3.0 the C parse alone matches the probe to within a few percent — 101% of it under Node 18.20.8 and 103% under Node 23.11.1, the excess over 100% being the difference between our build of the release and Node’s. There the discarded attempt is the OpenSSL parse, and it would cost the same with no JavaScript engine present. Second, the exception is small and nearly constant across runtimes: the Node-shaped error costs 4.781 μ s difference cannot be read directly as the price of the exception. What it does show is that a quantity of under Node 18.20.8 and 3.421 μ s under Node 26.10.0 (1.2% and 19.0% of the respective probes), and roughly half of it is stack capture (with capture disabled, 2.204 and 1.930 μ s). Disabling stack capture for the real probe saves 5.397 μ s and 1.767 μ s on the same two runtimes, the same order as a complete successful parse is spent on

the failing path before any exception exists, which is why we describe the component as the synthetic figure. Third, on runtimes bundling OpenSSL 3.5, where the parse is cheap, the three parts are of comparable size: under Node 26.10.0 the parse is 47% of the probe, the exception 19.0% and the binding 34.1%.

This is why the component is named a discarded *parse attempt* rather than as a discarded exception.

Section 2.1 records that we found no V8-specific baseline for Error construction cost against which to calibrate the remainder. Separating the two components requires either an instrument inside OpenSSL or a condition that reaches the same failure without constructing a JavaScript exception; we have neither, and we make no claim at that granularity. Where it is expensive the exception is about a hundredth of it, and even where it is cheap the exception is a fifth. These runtimes are Linux builds on x86_64; the ARM64 host figure of Table 1 was not itself decomposed this way, and its 18.04 μ s is of the same order as the OpenSSL 3.5 rows here, not the OpenSSL 3.0 rows.

4.3 The parts do not account for all of it

If the penalty for supplying a string were exactly the discarded attempt plus the conversion, the two would sum to it. They do not, quite:

observed penalty	23.92 μ s
discarded parse + conversion	18.62 μ s
residual	5.292 μ s

All three quantities are medians of per-round paired differences, per Section 3.1. The residual’s interval, [4.916, 5.501], excludes zero. The two measured parts

Table 2 The discarded attempt decomposed, per runtime, in μs (x86.64). *Probe*: `createPublicKey(secret)` inside Node, thrown and caught. *C parse*: the same OpenSSL calls issued from C against the OpenSSL release that runtime bundles, built from source. *V8 exception*: an `Error` of the same shape, message and properties thrown from 8 frames and discarded, with no OpenSSL call. *Binding*: the probe less the other two; its interval, from independent resampling of the three runs, is in the replication package. Medians of per-process medians over 15 processes (probe, exception) and 15 processes (C). The C column uses a stock build of each release, not Node’s own build of it, so the binding column is approximate (Section 10).

Runtime	OpenSSL	Probe	C parse	V8 exception	Binding
Node 18.20.8	3.0.16	390.5	394.4	4.781	-8.694
Node 20.20.2	3.0.19	443.4	431.3	4.796	7.292
Node 23.11.1	3.0.16	383.5	394.4	3.959	-14.88
Node 22.23.3	3.5.8	18.98	8.447	4.021	6.515
Node 24.21.0	3.5.8	19.54	8.447	4.136	6.952
Node 26.10.0	3.5.8	18.01	8.447	3.421	6.145

therefore account for 77.9% and not more; the remainder lies in the library’s own type and claim checking along the non-pre-parsed path. We report it rather than absorbing it into the headline, because a decomposition whose parts are declared to sum exactly to the whole has usually not been checked.

The pre-parsed path carries a smaller unexplained remainder of the same kind, and we record it for consistency rather than leaving the asymmetry unremarked: `verify()` with a pre-parsed key costs 6.042 μs against 1.292 μs of structural decode plus 2.166 μs of HMAC, leaving roughly 2.58 μs unattributed. Neither residual was measured directly; both are the subject of conditions we did not run, and both are stated as unattributed rather than assigned.

4.4 Confirmation without differencing

The decomposition reaches its conclusion by subtracting conditions — the same inference style this paper criticises elsewhere. It is therefore checked against V8’s sampling profiler, which attributes self time to named frames with no arithmetic

on our part. At 50000 iterations per run and 3 runs per environment, matched so the percentages are comparable, the asymmetric key parse accounts for a median 66.2% of self time on the host (range 65.3–67.4 across runs), 95.3% (95.1–95.6) under Node 18, and 61.5% (58.5–61.6) under Node 26. The frame does not appear among the top frames of the pre-parsed path in any environment.

Two methods, one answer. We rely on the ordering and the dominance rather than on the exact percentages: sampling attribution depends on rate and on what else the process is doing.

5 RQ2: What governs the magnitude

Table 3 varies the runtime with containerisation held constant, and Figure 3 plots the two columns that matter.

5.1 Engine major version does not distinguish the two slow runtimes

Node 18 and Node 20 ship V8 10 and 11 — different major versions of the engine —

Table 3 The same measurement across four container runtimes on one machine, 10 processes each of 20000 calls. The *Disc. parse*: the discarded asymmetric parse; *Pre-parsed*: `verify()` with a pre-parsed key. The host row is from Table 1 and is not containerised. [†]Inferred from the run timestamp, not recorded per row: that run predates the per-measurement OpenSSL field (Section 10).

Runtime	OpenSSL	V8	Disc. parse (μ s)	Pre-parsed (μ s)
Node 18 musl	3.0.16	10	373.79	7.58
Node 20 musl	3.0.19	11	420.62	7.25
Node 26 musl	3.5.7	14	22.94	6.56
Node 26 glibc	3.5.7	14	16.58	6.04
host, no container	3.6.3 [†]	14	18.04	6.042

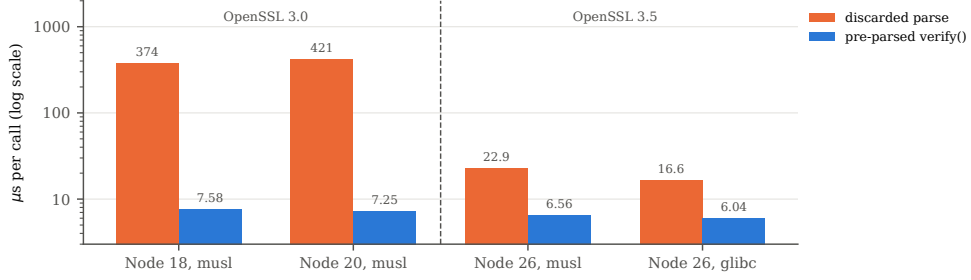


Fig. 3 The four container rows of Table 3 on a logarithmic axis. The discarded parse spans $25.36\times$ across images while the pre-parsed call spans $1.26\times$. The dashed rule marks the OpenSSL series boundary. Within this matrix it is not a controlled boundary, because the two fast images also carry a newer engine and a different build (Section 5.2); Section 5.5 controls it. The uncontainerised host row is omitted because its comparison with the container rows is confounded. Generated by `figures/make_figures.py` (Katha et al, 2026).

and share the OpenSSL 3.0 series. Both pay of order $373.79\text{--}420.62\ \mu\text{s}$. Node 26 ships OpenSSL 3.5 and pays $22.94\ \mu\text{s}$ on the same libc. Within the OpenSSL 3.0 series, then, changing the engine’s major version does not move the quantity; had Node 20 been fast, the attribution would have been to the engine.

5.2 What this design cannot separate

That argument establishes a negative within one OpenSSL series. It does not establish the positive attribution for the change between series, and we state the limitation rather than leaving it implicit.

The fast rows are Node 26. They carry OpenSSL 3.5 and V8 14 and every other

change in that build. OpenSSL version is therefore perfectly confounded with engine version and with build configuration across the 3.0–3.5 boundary. Nothing in this matrix can separate them. The absence of variation between V8 10 and V8 11 does not license the inference that V8 14 is also irrelevant; that is a different comparison, and we did not make it.

What the matrix supports on its own is the following, and no more: the magnitude co-varies with the bundled OpenSSL version across four images; the two slow images differ in engine major version and do not differ in magnitude; and the two libc variants of one image differ by less than the difference between series. Separating OpenSSL from the engine requires a crossed design — a

~~single Node version linked against both OpenSSLseriesruntime pairing a newer engine with an older OpenSSL, or the parse measured outside Node entirely—which we did not run. We therefore write. An earlier version of this paper had neither and wrote *co-varies with* throughout, including in the abstract and the conclusion, and we draw no causal conclusion about OpenSSL from this data.~~

~~The engineering evidence cited in Section 2.1 proposes a mechanism for such a regression. We do not adopt it, for the reason given there: it is not peer-reviewed and was not produced under a protocol we can assess. Section 5.5 reports both, and it is on them, not on this matrix, that the causal wording now used rests.~~

5.3 The effect is confined to key parsing

Across the four container rows, pre-parsed validation spans $1.26\times$, the HMAC primitive $1.39\times$ and structural decode $1.35\times$. The discarded parse spans $25.36\times$. The successful parse moves with it ($11.74\times$), so this is the cost of key parsing in general rather than anything peculiar to the failure path. The failure is simply the case in which an application pays that cost on every request for a result it discards.

5.4 What this design does not isolate

A reader may notice that the uncontainerised host row is of the same order as the Node 26 container row. That comparison is confounded and we draw no conclusion about containerisation from it: the host runs a different operating system from the containers, so operating system, C library and Node build all

change alongside the container boundary. Isolating containerisation would require a Linux-native baseline, which we did not obtain. The claims above concern only the four container rows, among which containerisation is constant.

5.5 Caused by OpenSSL: the parse without an engine

~~Table 4 removes Node entirely: the OpenSSL calls `createPublicKey()` makes for an HMAC secret, issued from C against 8 OpenSSL releases built identically.~~

~~In plain C the failure costs $394.4\ \mu\text{s}$ on OpenSSL 3.0.16 and $8.447\ \mu\text{s}$ on 3.5.8, a ratio of 46.69 [46.16, 46.90]. Nothing else differs between those two measurements: the program, the compiler, the flags, the machine and the protocol are the same. The magnitude is therefore *caused* by the OpenSSL release. That is a claim about this code path in these releases; it says nothing about OpenSSL performance generally.~~

~~The table also says where in OpenSSL the cost lives and when it went away. Almost all of it is the private-key parse — 99% of the full sequence on 3.0.16 — the step at which OpenSSL 3 hands the input to its provider-based decoders; the three public-key scans reject the input at the PEM header and are cheap in every release, and the error strings cost under a microsecond. The fall came in two steps: to $57.48\ \mu\text{s}$ in 3.1.8, and to $6.998\ \mu\text{s}$ in 3.2.6, after which it is flat to within a few microseconds through 3.6.1. We measured the steps, not the source changes that produced them; identifying those would take a bisection of the OpenSSL history that we have not done. A successful parse of real key material fell with it, which is why the effect is confined~~

Table 4 The failed asymmetric parse issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one compiler and one set of flags (x86_64), in μ s. Medians of per-process medians over 15 processes of 5000 calls. *Full*: everything `createPublicKey()` asks of OpenSSL for an HMAC secret, including the error-string drain; 95% intervals for it are in the replication package and are within 0.9% of the median for every release. Its parts follow. *Successful parse*: an RSA-2048 public key through the same path, for comparison.

OpenSSL	Full	Private-key parse	Public-key scans	Error strings	Successful parse
3.0.16	394.4	390.9	2.287	0.796	112.9
3.0.19	431.3	424.0	2.231	0.869	132.6
3.1.8	57.48	54.84	2.196	0.497	36.09
3.2.6	6.998	4.487	2.057	0.459	11.75
3.3.5	6.946	4.465	2.074	0.474	12.04
3.4.3	7.245	4.554	2.098	0.500	11.19
3.5.8	8.447	5.757	2.051	0.483	11.26
3.6.1	8.803	5.985	2.234	0.472	11.16

to key parsing in general rather than to the failure path.

Engine and library crossed.

The official runtimes of Section 3.5 contain the pairing stock container images lack: Node 22.23.3 ships V8 12.4.254.21 with OpenSSL 3.5.8, while Node 23.11.1 ships the *newer* V8 12.9.202.28 with the *older* OpenSSL 3.0.16. The probe costs 18.98 μ s on the first and 383.5 μ s on the second. The Node 18/20 pair of Table 3 showed that a change of engine at a fixed OpenSSL series leaves the cost where it was; this pair shows that a newer engine does not rescue an older OpenSSL. Together with the C measurement, the engine is excluded from both directions.

5.6 A second architecture

Table 5 repeats the runtime matrix, with the same images and scripts, on x86_64 (Section 3.7). Absolute values are not compared across the two machines; the question is whether the ordering holds.

The property the attribution rests on — every OpenSSL 3.0 image slower than every OpenSSL 3.5 image — holds on both architectures. The strict rank order does not: the two OpenSSL 3.0 images, Node 18 and Node 20, exchange places between ARM64 and x86_64. Their difference is small against the gap between series on either machine, and no claim in this paper depends on their relative order. On x86_64 the discarded parse spans 30.50 $\times$ across the images while pre-parsed validation spans 1.31 $\times$, against 25.36 $\times$ and 1.26 $\times$ on ARM64. The host decomposition of Table 1 was repeated too (Node 22.22.2, OpenSSL 3.5.5): the discarded parse is 19.42 μ s of a 36.48 μ s call (53%), the symmetric constructor 1.261 μ s and the HMAC primitive 3.105 μ s, so the component ordering of Table 1 holds there as well. The full x86_64 host table is in the replication package.

Table 5 The runtime matrix of Table 3 on two architectures, same images and scripts, in μs . ARM64: 10 processes of 20000 calls. x86_64, a shared cloud virtual machine: 10 processes of 20000 calls. Absolute values are not comparable across the two machines.

Image	OpenSSL	Discarded parse		Pre-parsed <code>verify()</code>	
		ARM64	x86_64	ARM64	x86_64
node:18.20.8-alpine	3.0.16	373.79	607.1	7.58	12.66
node:20-alpine	3.0.19	420.62	556.3	7.25	11.89
node:26.6.0-alpine	3.5.7	22.94	42.13	6.56	10.94
node:26.6.0-bookworm	3.5.7	16.58	19.91	6.04	9.701

Table 6 In-situ validation cost with a string secret, by arrival rate. The two columns are the ascending and the descending pass over the same seven rates on one process; each is a single window, and the pair is two observations rather than an interval. Host load average during each window is committed with the measurement.

Arrival rate (requests/s)	Mean per call (μs)	
	ascending	descending
5	928.24	1027.97
10	976.28	1017.34
25	918.54	934.47
50	758.57	737.12
100	571.59	607.78
200	494.91	532.50
400	452.82	467.62

6 RQ3: Deployment and prevalence

6.1 In-situ cost across arrival rates

The deployed service was driven at seven arrival rates, each measured by the paired-scraps difference of Section 3.8, with the whole sweep run ascending and then descending on one process. Table 6 reports the range across the two passes at each rate for the configuration the service deploys, a string secret, and Figure 4 plots it.

One observation follows, and it is the one this section claims: the cost is present

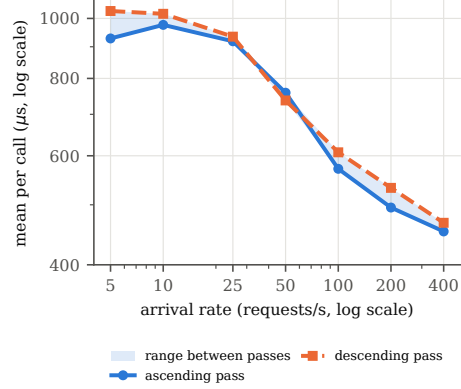


Fig. 4 In-situ validation cost against arrival rate, string secret. The two passes are drawn separately rather than averaged: Section 6.2 reports that they differ systematically, and averaging would conceal that. The shaded band is the range between the passes, which is two observations rather than an interval. Generated by `figures/make_figures.py` (Katha et al, 2026).

in deployment and is far larger than the isolated host figure of Table 1, consistent with Section 5 in that the deployed runtime is the one carrying the older OpenSSL.

Two things visible in Table 6 ~~are not~~ elaimed need care, and we set them out explicitly.

Wall-clock exceeds the prediction; processor time does not.

The deployed runtime’s discarded parse costs $373.79\ \mu\text{s}$ (Table 3), and the remainder of the validation path is of order $7.58\ \mu\text{s}$, giving an expected total near $381\ \mu\text{s}$. The ~~observed figures wall-clock means observed across the sweep~~ are $453\text{--}1028\ \mu\text{s}$, ~~exceeding that by $71\text{--}647\ \mu\text{s}$~~ . Contention inflates a duration rather than deflating it, so these are upper bounds ~~and are not inconsistent with the microbenchmark; but a gap of roughly μs is unaccounted for, and we report it as unaccounted rather than attributing it. It was not decomposed, and Section 6.4 shows where the excess is not: measured as processor time, the validation call costs within 0.8% of the prediction. The excess is therefore time the call spent off the processor on a shared host, not additional work. We did not decompose that off-processor time further, and no claim below depends on it.~~

The decline with arrival rate is not interpreted.

Cost falls monotonically from $928.24\ \mu\text{s}$ at 5 requests/s to $452.82\ \mu\text{s}$ at 400 requests/s. This is the opposite of what contention alone would produce, and several mechanisms unrelated to validation would produce it — a fixed per-window overhead amortising over more calls, processor frequency scaling and idle-state exit latency at low rates, cache and branch-predictor residency, or continued tier-up in the engine. We did not run the conditions that would distinguish them. We therefore report the numbers and do not claim that validation cost depends on arrival rate; establishing that would require varying window duration at fixed rate, pinning the frequency governor, and extending

warmup at low rates, none of which we did.

6.2 The two passes differ systematically

Sweeping up and then down is not only replication; it detects warm-up, thermal and hysteresis effects. If the difference between passes were noise, its sign would alternate across rate points. It does not. In the string-secret sweep the descending pass is slower at 6 of 7 rate points; in the pre-parsed sweep the descending pass is *faster* at 7 of 7. A consistent sign across seven points is not what noise produces.

This has a direct consequence for how the ranges in Table 6 should be read. A systematic component means the two passes are not independent samples of a common value, so the range between them understates uncertainty further than a range of two already does. We therefore report ranges and draw no inference that depends on their width. It is also a methodological finding in its own right: a single-direction sweep would have produced a curve carrying a consistent bias with no way to detect it.

6.3 Host load, not the condition, drives the outliers

Agreement between passes varies considerably across windows. We distinguish *relative* disagreement, the gap between passes as a fraction of their mean, from *absolute* disagreement, the gap in microseconds; the two behave differently and the distinction carries this subsection.

The widest relative disagreement in the dataset and the tightest occur at the same arrival rate in different sweeps. The wide one is the highest-load window in its own sweep on the descending pass,

though not on the ascending pass, whose maximum load falls at a different rate. The tight one, at the same arrival rate in another sweep, sits at a lower load on both passes and its two passes agree to within $0.02 \mu\text{s}$. Relative disagreement therefore does not track the condition under measurement.

We considered two alternative explanations for the relative asymmetry and reject both. A small-mean effect would require absolute disagreement to be roughly constant, so that its relative size grew only as means shrank; absolute disagreement instead varies over three orders of magnitude and is largest where the means are largest, which is the opposite pattern. A measurement floor would require the affected values to sit at the instrument’s resolution; they do not, and in any case the means reported here are computed from the histogram’s sum and count rather than from its buckets, so bucket resolution cannot bound them.

The operative consequence is that in-situ figures are reported with the host load recorded during their window, and comparisons between windows at different loads are not treated as comparisons between conditions.

6.4 Corroboration: the cost is processor time, and it matches the prediction

An in-situ figure of this magnitude has two readings. Either it is work the service performs, or it is ~~an artefact of the surrounding measurement time the request spends waiting — ambient handler cost that would be there without validation, or a property of the instrument rather than of the code queued behind other requests, descheduled by a contended host, or inflated by the~~

Table 7 Validation enabled against validation disabled, at one arrival rate, two 180 s windows on the same pod and image, 36001 requests each. A signed token is on the wire in both arms; the disabled arm removes only the handler’s `jwt.verify()` call. Application and sidecar CPU are means of one-second cgroup samples in the window; CPU per call is that mean divided by the achieved rate. Event-loop lag is the mean of the per-scrape p99. Host load is a single sample at window start.

	disabled	enabled
Achieved rate (req/s)	200.006	200.006
Application CPU (millicores)	69.45	146.38
CPU per call (μs)	347.3	731.9
Sidecar CPU (millicores)	53.40	49.13
Event-loop lag (ms)	3.137	3.159
Client mean latency (ms)	1.6802	2.0332
Host load, 1 min	3.53	5.11

instrument. We separate them with an A/B of the same service at one arrival rate, with validation enabled in one window and disabled in the next ~~under otherwise identical conditions, on the same pod running the same image, and read the processor time the application container was charged from its cgroup accounting rather than from a clock.~~ Table 7 reports it.

~~Removing — validation — removes Enabling validation adds $384.6 \mu\text{s}$ per call, % of the enabled mean. The arrival rate, the request path and the rest of the handler are unchanged between the arms; only the validation call is withdrawn. The in-situ figure is therefore attributable to that call rather than to ambient handler cost or to the instrument, which is what this table is for.~~

~~Whether the time is spent executing or spent descheduled inside the call is a further question of processor time per call. The runtime matrix predicts $381 \mu\text{s}$ for the same work on the same runtime (Section 6.1); the two agree to within 0.8%. They are obtained by instruments that share no code: cgroup~~

~~CPU accounting on a deployed pod, and a wall-clock difference does not answer it. Three observations together do. The isolated microbenchmark of Section 4 measures the same operation in a single process with no concurrent load at all and finds it dominated by the same component. The sampling profiler of Section 4 attributes a median % of *self* time to that frame, and self time is time on processor. And the A/B above shows the cost survives into deployment and belongs to the call. The microbenchmark and the profiler establish that the work is processor time; the A/B establishes that the deployed figure is the same work and not something else.~~

~~single-process microbenchmark in an isolated container. A request that queues or is descheduled accrues wall-clock time but no CPU, so this figure cannot contain queueing delay.~~

~~That is the whole of what this table establishes, and we are explicit about what it does not. It does not reconcile the in-situ magnitude with the microbenchmark: Section 6.1 reports a gap of roughly μ s between the observed means and the sum of the components measured on the deployed runtime, and one A/B at one rate does not decompose it. It does not bound the contribution of host load, because the load figures are window-start snapshots rather than means. And Three observations weigh against reading the difference as an artefact of the two windows. The sidecar, in the same pod under the same contention, did not rise with the application: 53.40 millicores disabled against 49.13 enabled. The one-second CPU samples of the two arms do not overlap at all — disabled never exceeded 100.577 millicores in 127 samples, enabled never fell below 114.963~~

~~in 127 — which separates the arms without a distributional assumption. And event-loop lag is unchanged, so neither arm was near saturation. The two windows are two observations, not a sample: the difference carries no interval, and we compute none from it.~~

~~An in-situ figure of this magnitude has two readings. Either it is work the service performs, or it is an artefact of the surrounding measurement — ambient handler cost that would be there without validation, or a property of the instrument rather than of the code. The second is excluded by an A/B of the same service at one arrival rate, with validation enabled in one window and disabled in the next under otherwise identical conditions; the two runs are committed with the rest of the measurements in the replication package. Withdrawing the validation call, with the arrival rate, the request path and the remainder of the handler unchanged, removes the great majority of the per-call cost, which locates it in that call.~~

~~Whether the time is spent executing or spent descheduled inside the call is a further question that this comparison does not answer. Two instruments do. The isolated microbenchmark of Section 4 measures the same operation in a single process with no concurrent load at all and finds it dominated by the same component, and the sampling profiler of Section 4 attributes a median % of *self* time to that frame, self time being time on processor.~~

6.5 How common is the avoiding pattern, in our sample?

Three instruments, three frames, reported separately as Section 3.9 requires. All counts were captured on 2026-09-18. None

of the figures below is a population proportion.

Population co-occurrence.

Of 790528 public JavaScript files importing the library through `require`, 29 also mention the pre-parsing interface; for JavaScript through `import` the figures are 524288 and 38; for TypeScript through `require`, 8288 and 0; and for TypeScript through `import`, 333824 and 101. Cells are not pooled into a rate: a file using both import forms would be counted twice, so their sum 1656928 is an upper bound on the union and the largest cell 790528 a lower bound. No cell exceeds 0.0303% of files mentioning the interface. These are search-engine estimates of file counts over public repositories only, they measure co-occurrence rather than use, and the denominators are returned *rounded* — each is an exact multiple of 1024 — so they should be read as magnitudes rather than as counts established file by file.

Classified call sites.

Of 327 call sites classified across 247 distinct repositories in our sample, 0 pass a pre-parsed key. Environment variables account for 176 of those call sites, untraced identifiers and member expressions 93, string literals 52, file contents 4 and buffers 2. No file in this corpus contains the pre-parsing interface at all, in any spelling, so the zero reflects the pattern's absence from the sample rather than a classifier that cannot see it. Incidentally, 309 of 327 call sites (94%) pass no algorithm restriction, which Section 8 returns to.

An earlier version of the classifier recognised only the conventional `jwt.verify()` spelling, which filtered the corpus toward one import idiom. It was re-run over the previously unclassified

residue with a matcher that resolves the local binding of the import — covering arbitrary aliases, namespace imports, destructured and renamed imports, and direct calls on the `require` expression. That recovered 12 further call sites, included in the totals above, of which 0 pass a pre-parsed key.

Targeted counter-search.

A sample can only show a pattern is rare if it could have found it. Every file co-occurring the library with the pre-parsing interface was therefore fetched: 191 files, of which 120 contain a validation call, spanning 112 distinct repositories. An automated pass flagged 13 call sites as passing a pre-parsed key and all were then read by hand: 5 are false positives — projects defining their own helper function of the same name, one of which returns a string — 1 could not be resolved within its file, and 7 are genuine, across 6 distinct projects.

The rate, with its denominator.

Among the 112 repositories that both reference the pre-parsing interface and call the validator, 6 use it: 5.36%. That denominator is the most favourable available, since those projects demonstrably know the interface exists, and it is not the population. Within this frame the pattern is rare rather than absent, and it is rare even among projects already aware of it. We do not extrapolate beyond the frame.

7 RQ4: One library, or a design choice?

A cost found in one library is an observation about that library. It becomes a statement about a class of defect only if the mechanism can be named independently of the library and

predicts which other libraries have it. The mechanism here can be named: *resolving key material of unknown type by attempting the asymmetric parse first, and treating its failure as the signal that the material is a shared secret*. Table 8 tests the prediction against six further libraries.

What the source says.

Only `jsonwebtoken` attempts the asymmetric parse. The others resolve the ambiguity in one of three ways, none of which involves a parse that can fail on a shared secret. *Inspect, then dispatch*: `fast-jwt` examines the text of the key — whether it carries a PEM header or is a JSON Web Key — and calls exactly one constructor, skipping even that inspection when every permitted algorithm is HMAC; `PyJWT` likewise inspects the text, and rejects PEM- or SSH-looking material offered as an HMAC key rather than trying to parse it. *Refuse the ambiguous input*: `jose` does not accept a string for an HMAC key, and `golang-jwt` accepts only a byte slice for HMAC verification, returning a key-type error for anything else. *Type the key*: `jjwt` requires a `SecretKey` for HMAC and a `PublicKey` for signatures, so the distinction is made by the Java type system; `nimbus-jose-jwt` offers a constructor that takes a string and converts its bytes without parsing them. Refusing a string does not by itself remove per-call work, however: `jose`, handed raw bytes, imports them into a `WebCrypto` key on every call.

What the measurements say.

Of the seven libraries, two pay for being handed their raw form rather than a key object, in the sense that the paired interval on the penalty

excludes zero; for the other five it does not, and their point estimates are within a microsecond of zero. The two are `jsonwebtoken` and `jose`, and they pay for different reasons. `jsonwebtoken` performs the failing asymmetric parse: its penalty is 465.9 μ s under OpenSSL 3.0.19 and 24.56 μ s under OpenSSL 3.5.8, a string-to-pre-parsed ratio of 46.55 and 3.48 respectively. `jose` parses nothing that can fail, but imports the raw bytes into a `WebCrypto` key on every call: 42.66 μ s and 24.80 μ s under the same two runtimes. Both penalties grow on OpenSSL 3.0, so the OpenSSL release sets the price of per-call key handling generally; but the failing parse is the costliest form of it, about 11 times `jose`'s on OpenSSL 3.0. Two of the libraries that pay nothing resolve the key once, when a verifier or parser is built (`fast-jwt`, `jjwt`), so a raw secret costs nothing per call by construction; that is itself the design point.

What this establishes.

The libraries without the problem matter as much as the ones with it. All seven validate the same kind of token with the same primitive — against the same OpenSSL, in the JavaScript cases — and five pay nothing for a raw secret. The cost therefore follows a design choice, resolving key material on every call, and not JSON Web Token validation, HMAC or OpenSSL as such: OpenSSL 3.0 sets the price, but only a library that makes that choice is charged it, and one that makes it by way of a failing parse is charged the most. The failing parse is exactly what the order-of-evaluations profiler of Selakovic et al (2017) is designed to find, and the alternatives above are the fixes that line of work suggests: reorder the test, make it cheap,

Table 8 HS256 verification across seven JWT libraries in four languages (x86_64), in μs . *Asym. first?*: given a string HMAC secret, does the library attempt to parse it as an asymmetric key, as read from its source. *String* and *pre-parsed*: the secret in the form the library’s documentation shows, and in the library’s own key object built once (Section 3.6). *Penalty*: median of per-round paired differences, with a 95% bootstrap interval over rounds. Medians of per-process medians over 15 processes of 20000 calls. Absolute costs measure different amounts of work in different runtimes and are not a ranking of libraries. ^aNo string accepted for an HMAC key; the rawest accepted form (bytes) is used. ^bA string is accepted only by a deprecated setter that resolves it once, when the parser is built.

Library	Lang.	Version	Crypto backend	Asym. first?	String	Pre-parsed	Penalty	95% interval
jsonwebtoken	JS	9.0.3	OpenSSL 3.0.19	yes	476.2	10.23	465.9	[465.2, 467.8]
jsonwebtoken	JS	9.0.3	OpenSSL 3.5.8	yes	34.22	9.844	24.56	[24.07, 24.82]
fast-jwt	JS	6.3.3	OpenSSL 3.0.19	no	7.564	7.570	0.070	[-0.116, 0.111]
fast-jwt	JS	6.3.3	OpenSSL 3.5.8	no	7.917	7.934	-0.068	[-0.134, 0.197]
jose	JS	6.2.12	OpenSSL 3.0.19	no ^a	121.2	78.42	42.66	[38.50, 45.89]
jose	JS	6.2.12	OpenSSL 3.5.8	no ^a	98.89	73.98	24.80	[21.91, 26.51]
PyJWT	Python	2.15.0	hashlib, OpenSSL 3.0.13	no	41.33	41.28	-0.103	[-0.457, 0.338]
golang-jwt	Go	5.3.1	Go crypto/hmac	no ^a	4.810	4.764	0.034	[-0.042, 0.117]
nimbus-jose-jwt	Java	10.10	JCA SunJCE	no	4.358	4.229	0.304	[-0.098, 1.016]
jjwt	Java	0.13.0	JCA SunJCE	no ^b	12.15	12.33	-0.406	[-0.715, 0.648]

or move it off the per-call path. The comparison does not estimate how many libraries make the choice; it shows that making it is what matters.

8 A fix, and a constraint on it

The fix is to dispatch on the token’s declared algorithm before attempting the asymmetric parse, so that an HMAC token with an ordinary string secret is resolved directly. Applied, this takes the call from 29.92 μs to 7.208 μs , a factor of 4.15, and to within 1.208 μs of supplying a pre-parsed key, with no change in calling code. That residual is the median of the per-round paired differences, not the difference of the two medians; the two are not identical and the former is reported throughout.

We tested it against the library’s own suite rather than only against assertions of our own. Cloned at tag v9.0.3, the patch applies cleanly and the suite is unchanged: 511 passing and 0 failing unpatched, 511 and 0 patched.

These figures are from the host. The practical benefit is largest where the discarded parse is largest — of order

373.79 μs on the runtime the service actually deploys (Table 3) — and we did not measure the patched library on the container runtimes. The reported factor of 4.15 is therefore a lower bound on the improvement available in the deployed environment, and we state it as a host result rather than as a general one.

The constraint, which is the more important half.

The discarded attempt is not purely wasteful. The *success* of that parse on asymmetric key material is what produces a key object whose type a later check depends on, and that check is what prevents symmetric and asymmetric material from being interchanged (Sheffer et al, 2020). A change that dispatches on the declared algorithm while relaxing which key material may take the fast path alters which inputs reach that check.

The fix must therefore be narrow: the fast path is restricted to string secrets that cannot be asymmetric key material, so anything that might be resolves exactly as it does today. Removing that material restriction — the form of the optimisation we argue against — produces 2 failures in the library’s own suite, both in

its key-confusion tests (“should not allow HMAC verification with an RSA key in PEM format” and, under “signing with pub key as symmetric”, “should not verify”). All of these outcomes — the suite unchanged under the narrow patch, two key-confusion failures without its guard — reproduce on the x86.64 machine of Section 3.7 under a different Node and OpenSSL release (Katha et al, 2026).

A reduction of our own contribution.

We had written a regression test for this property and intended to offer it upstream. Running the library’s existing suite showed that it already constrains the change: a contributor proposing the unrestricted optimisation would be caught by tests they are required to run. We therefore withdraw that contribution. What remains is narrower and, we think, still worth stating: the cost documented in Section 4 cannot be removed by the obvious route, and the reason is a property the maintainers already test for but which is not evident from the code being changed.

The independent defence against this class of confusion is for the caller to restrict permitted algorithms (Sheffer et al, 2020), and 94% of the call sites surveyed in Section 6.5 do not.

Reported upstream.

Before reporting, we searched the library’s issue tracker for earlier reports. One exists: an issue opened in April 2024¹ reports that HS256 verification in the 9.x series is 34 times slower than in 8.5.1, on a Node release that bundles OpenSSL 3.0. It diagnoses no cause and, at the time of writing, has no maintainer response. That is the symptom this paper explains, observed

independently two years earlier by a user who could not see its source. We reported the mechanism, with a minimal reproduction, per-runtime measurements and the OpenSSL finding, as a separate issue², and a pull request implementing the narrow form of the fix, with its guard, is open against it. At the time of writing neither has received a maintainer response. We state that rather than omit it: the library is maintained at a slow cadence and a response may postdate publication.

9 Implications for practice

Pre-parse key material once, at startup.

On the host this removes 23.92 μ s per validation [23.58, 24.00]; on the runtime the service actually deploys, where the discarded parse costs 373.79 μ s, it removes far more. It requires no architectural change, no new component and no movement of a trust boundary.

Treat the base image as part of the measurement.

The same code, the same library and the same architecture differ by $25.36\times$ on this path across the container images measured, because they pin different OpenSSL versions. A performance figure quoted without the base image it was obtained on is not reproducible, and a regression appearing after a base-image bump may have nothing to do with the application. ~~This holds whether or not the OpenSSL version~~ Section 5.5 shows that on this path the OpenSSL release is the cause, ~~which Section 5.2 states we cannot establish;~~ but the practical point would

¹<https://github.com/auth0/node-jsonwebtoken/issues/966>

²<https://github.com/auth0/node-jsonwebtoken/issues/1046>

hold even if it were not, since it is enough that the image changes the number.

For library authors: do not use a failing parse as a type test.

Section 7 found three designs that avoid the cost — inspect the material and dispatch, refuse ambiguous input at the interface, or type the key — and all three ship in maintained libraries today. What they share is that resolution happens once, or not at all, rather than on every call. The design that pays most is the one that asks an expensive parser to answer a cheap question on every request.

Pin the algorithm.

Independently of anything here, callers should restrict permitted algorithms (Sheffer et al, 2020). Most of the call sites we surveyed do not, which leaves them relying entirely on library-internal key typing for a property they could assert directly in one argument.

9.1 Relocation is not the first remedy

~~Because a A cost believed to be cryptographic invites relocation, we implemented that relocation and report what it does. This subsection rests on source inspection and a manual observation; it carries no measured result, and we set out below why we report none.~~

~~Applying a mesh `RequestAuthentication` policy causes the sidecar to validate the token before proxying. It does not cause the application to stop validating: verification is *duplicated*, not relocated, and no work leaves the event loop. Making relocation effective therefore to a sidecar or gateway. The cost measured~~

~~here is not cryptographic and is largely avoidable in place, so an architectural remedy that also moves a trust boundary should not be the first thing reached for. We implemented the relocation the earlier design proposed; applying a mesh validation policy duplicates verification rather than moving it, and making it effective requires the application to trust something the sidecar tells it. In our implementation the handler reads a sidecar-populated claims header and, when it is present, returns the protected payload without verifying anything a proxy-supplied claims header — the branch is a single conditional in `service-a/index.js` in the replication package.~~

~~Trusting an identity header injected by an upstream proxy, without stripping any client-supplied copy at the trust boundary and without a fail-closed path when the policy is not applied, is a known deployment a known anti-pattern rather than a novel finding, and we report it as such. In our implementation the sidecar overwrites the header only while a policy is applied; in any other state nothing sanitises it. With the guard filter applied, a request carrying a forged claims header and no credential is refused. With the guard removed, the same request returns the protected payload. We reproduced this sequence manually — refused, served, refused — and it is consistent with the note recorded alongside the affected runs that needs a filter stripping client-supplied copies. We report that only as a design note: it carries no measurement, and the configuration, the observation and the inconclusive automated probe are documented in the replication package. The remedy is an always-applied filter that strips any client-supplied copy of~~

~~the header on entry to the pod, before any validation filter runs, matched on the connection manager rather than on the validation filter, because in the state that needs protection no validation filter exists.~~

~~We wrote an automated probe to turn the observation into a table and it does not produce a trustworthy one. The mesh applies a configuration change asynchronously, on a timescale comparable to the probe interval, so whether a probe observes the guarded or the unguarded behaviour depends on whether the proxy had converged when the request fired. Two runs of the same script produced contradicting values for the same configuration. Rather than report the run that agreed with our expectation, we report none: the script’s own negative control declared both runs inconclusive, and we retain it in the replication package with that limitation recorded. Making this measurable requires gating each probe on *observed* configuration convergence—polling until repeated identical responses—rather than on a fixed delay. We have not implemented that, and until it is done this subsection should be read as a mechanism argument supported by an observation, not as a measurement.~~

~~The narrow point that survives is the one this paper’s measurements support: the cost documented in Section 4 is largely avoidable in place, so an architectural remedy that also moves a trust boundary should not be the first thing reached for.~~ (Katha et al, 2026).

10 Threats to validity

~~Every measurement comes from a single~~

Two architectures, one of them shared.

The primary measurements come from a single ARM64 host and its container runtimes under one engine. We have not verified any of this on host. Section 5.6 repeats them on x86_64. The mechanism is a property of a library’s API contract—an asymmetric parse attempted before a symmetric one—64, and there the split between OpenSSL series and the component ordering of the host decomposition hold. But the x86_64 machine is a virtual machine on shared cloud infrastructure, whose absolute timings are known to be unreliable (Laaber et al, 2019); we therefore take only orderings and ratios between interleaved conditions from it, with and no magnitude in this paper should be read as holding on x86_64 hardware in general. A repetition on a dedicated x86_64 host, which the replication package scripts in one command, would turn the ordering check into a replication of magnitudes. The C probe, the exception baseline and the cross-library comparison were measured only on the failure discarded—and of the cost of key parsing in a particular OpenSSL series, and neither is a property of an instruction set. But an expectation is not a measurement, and the magnitude in particular depends on how OpenSSL’s key parsing performs on a given platform, which is exactly the kind of quantity that varies with architecture. No claim here should be read as holding beyond x86_64 machine. Their conclusions rest on ratios between conditions measured in the same rounds, which is the kind of quantity a shared machine can support, and the effects they report are far larger than the machine’s noise: the median between-process coefficient of variation is 1.5%, 2.9% and 2.8% in the three runs. The largest values (28.0%, 27.7%

and 20.2%) occur on sub-microsecond or JIT-dominated conditions that carry no claim, and medians of per-process medians are insensitive to them. The C probe needs no JavaScript runtime and the scripts rebuild every OpenSSL release from source, so repeating it on the ARM64 host or a dedicated x86.64 machine is one command.

~~As Section 5.2 states, the fast rows carry a newer OpenSSL, a newer engine and a different build simultaneously. The matrix establishes that the magnitude co-varies with the OpenSSL series and that engine major version does not distinguish the two slow runtimes; it does not establish that OpenSSL is the cause of the difference between series. A crossed design would settle it and we did not run one.~~

The C probe reproduces Node's calls, not Node's build.

Section 5.5 issues the OpenSSL calls Node issues, as read from the Node source, against OpenSSL releases we built. Node builds its bundled OpenSSL with its own configuration, so the C figures approximate rather than reproduce the parse cost inside a given runtime, and the binding column of Table 2 inherits that approximation — it is slightly negative on two OpenSSL 3.0 runtimes for that reason. The causal claim does not inherit it: it compares releases under one build configuration with nothing else varying.

~~The dominant component is a discarded parse attempt together with the exception that terminates it. We report the pair. Section 4.2 gives the one bound our data supports and states why we cannot do better without an instrument inside OpenSSL~~

The exception baseline is a reconstruction.

Node constructs the error from C++ through V8's API; our baseline constructs an error of the same shape, message and properties from JavaScript at a comparable stack depth. The two paths share V8's error construction and stack capture, and the paired stack-capture measurement on the real probe agrees in order of magnitude with the synthetic one, but they are not the same code path. The ARM64 host figure of Table 1 was not itself decomposed; Table 2 decomposes the same component on six other runtimes.

The headline host figure has no per-row OpenSSL version.

The host decomposition of Section 4 predates the per-measurement OpenSSL field, ~~and the host's OpenSSL has since moved by a patch release, so the version that produced those figures is known by run timestamp rather than from the measurement record.~~ The version in force is known from the run timestamp and a note recorded at the time: OpenSSL 3.6.3, which a package upgrade later that day moved to 3.6.4 under the same Node binary. We report it as inferred, marked as such in Table 3 ~~therefore leaves the host OpenSSL cell unrecorded rather than filling it with a version captured later.~~ Since Section 5 shows this quantity varies $\times$ with the runtime, this is a material limitation on the headline figure and not a bookkeeping detail: the, rather than as recorded. Three measurements that do carry their version bound how much this matters. The same probe costs 18.01 μ s under Node 26.10.0 with OpenSSL 3.5.8, 19.42 μ s in the x86.64 host repetition with OpenSSL 3.5.5, and

~~16.58 μ s is attributable to the host as configured at that time and to nothing more specific. The container rows, which carry the attribution of Section 5, each record their own in the Node 26 container with OpenSSL 3.5.7 — all of the same order as 18.04 μ s, and all from OpenSSL releases after the step of Table 4. The headline figure is therefore representative of a post-3.2 OpenSSL, which is all the paper uses it for; it would be wrong by more than an order of magnitude only if the host had run an OpenSSL 3.0 or 3.1 release, which the record excludes.~~

Ranges of two, with a systematic component.

The in-situ dispersion is the range across an ascending and a descending pass. It is not a confidence interval, and Section 6.2 shows the two passes differ systematically rather than randomly, so they are not independent samples of a common value and the range understates uncertainty further. No inference in this paper depends on the width of those ranges.

One in-situ process, two passes.

The ~~whole deployment result rate sweep~~ rests on one process sweeping seven rates twice. That is sufficient for the claim made — the cost is present in deployment and of this order — and insufficient for any claim about the shape of the cost-versus-rate relation, which Section 6.1 accordingly does not make.

~~The observed — μ s exceeds the sum of the measured components on the deployed runtime by a margin we did not decompose, and~~ The magnitude claim does not rest on the sweep alone: the A/B of Section 6.4 is a separate pair of windows, measured with a separate instrument, and it agrees with the microbenchmark prediction on processor

time. The wall-clock excess over that prediction is attributed to time off the processor and not decomposed further; the decline with arrival rate has several candidate explanations unrelated to validation that we did not test — (Section 6.1 ~~states both as unaccounted rather than interpreting them~~).

Sampling frames are not interchangeable.

The three prevalence instruments of Section 3.9 have different units and different selection mechanisms, and none supports a population proportion. The co-occurrence counts measure two strings appearing in one file, with a false-positive rate established at 5 of 13 on hand inspection. The classified sample has unknown inclusion probabilities, a limitation of repository search noted more generally by ~~Kalliamvakou et al.~~ Kalliamvakou et al (2014). The counter-search is selected on the outcome.

The matcher’s coverage.

Call sites are found by resolving the local binding of the import, which covers aliases, namespace imports, destructured and renamed imports and direct calls on the require expression. It does not resolve bindings across files, so a project that constructs a pre-parsed key in one file and validates in another is invisible to this analysis. This is the strongest threat to Section 6.5; within-file tracing found no such case, which is weak evidence against it.

Host load, and the confound that motivated recording it.

A throughput curve measured on a host shared with its own load generator moves with unrelated activity on that host, as Section 3.10 describes, which is why no

capacity claim appears in this paper. Every in-situ window here carries the load average observed during it, and Section 6.3 shows that window-to-window agreement tracks that load rather than the condition. Contention inflates a duration rather than deflating it, so in-situ figures are upper bounds; but comparisons between windows at different loads are not comparisons between conditions, and we do not treat them as such.

~~Every figure in-~~

The A/B is two windows at unequal load.

~~The two arms of Table 7 comes from two windows whose are consecutive windows rather than interleaved rounds, and their host load is known only at the instant each began. Section 6.3 shows that window-to-window agreement in the rate sweeps tracks load rather than the condition under measurement, so a load excursion inside either A/B window would be invisible here. Contention inflates a duration rather than deflating it and the enabled arm is the slower one, so an unobserved excursion in that window would inflate the difference rather than manufacture it: the direction of the conclusion is safe, its magnitude is not. That bars comparing them on wall-clock time, and we do not. It does not bar the CPU comparison the section rests on: contention inflates elapsed time, whereas a container's own cgroup accounting charges it only the processor time it used. Attaching the per-second sampler used elsewhere in the instrumentation host sampler to a re-run of both arms would remove this interleaved, would remove the limitation; we did not. The two arms are also separate windows rather than interleaved rounds, so the design is weaker on this~~

~~one comparison than the protocol of Section 3.1 requires elsewhere.~~

Token caching is not in the comparison.

A service that caches verified tokens pays this cost once per token rather than once per request. We did not measure such a configuration, so the per-request figures here do not describe a caching deployment.

One rater.

The 13 automatically flagged call sites were read and classified by the first author, and the authors are not disinterested: the paper's claim is stronger the more of them are false positives, and 5 were classified that way. There was no second rater and no inter-rater agreement to report. The per-case verdicts, the evidence for each and the hashes of the files they were read from are published so the adjudication can be repeated, which is weaker than having repeated it.

~~The mechanism is specific to one JavaScript implementation~~

Seven libraries, one version each.

Section 7 covers seven libraries in four languages, chosen as the most used in each ecosystem, each at one release. Whether other implementations or other language ecosystems attempt key resolution in the same order is unmeasured, and the paper's framing — that cryptographic cost is over-attributed — is therefore demonstrated for one library rather than established as a class of defect. Other libraries, and other releases of these, may resolve keys differently; each classification is from source at the version measured. The comparison supports the claim that the cost follows a design choice rather than

JWT validation; it does not estimate how many libraries make that choice.

11 Conclusion

The per-request cost of token validation in this library is real, is worth attending to, and is not the signature: the primitive is $2.208\ \mu\text{s}$ of a $29.92\ \mu\text{s}$ call. It is not key conversion either, which costs $0.583\ \mu\text{s}$. It is a parse attempted in the wrong order, whose failure is constructed and thrown away on every request at $18.04\ \mu\text{s}$, 60% of the call. ~~We report that quantity as the parse attempt together with~~ Where that attempt is expensive it is almost entirely OpenSSL work; the exception that ends it ~~, because we did not separate them costs a few microseconds.~~

Its price ~~co-varies with the OpenSSL version—is set by the OpenSSL release~~ a base image happens to pin, varying $25.36\times$ across runtimes while the rest of the validation path varies by $1.26\times$. ~~We state that as a co-variation because the available runtimes confound the OpenSSL series with the engine version shipped alongside it— and set causally, not only by association: the same calls from C cost $394.4\ \mu\text{s}$ on OpenSSL 3.0.16 and $8.447\ \mu\text{s}$ on 3.5.8, and a newer engine paired with an older OpenSSL is the slow runtime, not the fast one.~~

It is a design choice, not a property of JWT: of six other libraries in four languages none attempts the parse, five pay nothing for a raw secret, and the one that re-imports its key on every call pays an order of magnitude less. The symptom had been reported upstream two years before this study without a diagnosis; we have reported the mechanism.

The pattern that avoids the cost is rare in our sample: among 112 repositories that already reference the pre-parsing interface and call the validator, 6 use it. No population proportion follows.

Two explanations that present themselves before the measurement are refuted by it. That the dominant cost is key conversion is refuted by measuring the conversion. That a throughput collapse under layered enforcement characterises the service is refuted by the collapse point moving with unrelated host activity, which is why no capacity claim appears here. Both have the same shape, and it is the shape this paper’s methodology is built to avoid: a cost attributed to the variable under study while a second variable moves alongside it, unrecorded at the granularity at which it could change. The bundled OpenSSL version in Section 5 is the same hazard caught in time — the quantity varies $25.36\times$ with a component most measurement protocols record once per run, if at all. The lesson is not that one should measure in deployment. It is narrower: record the identity of the component whose behaviour is being measured, on every measurement, at the granularity at which it can change.

Data availability. The data supporting the results of this study are openly available in the ~~archived~~ replication package ~~, deposited on Zenodo under the DOI given in that reference~~ (Katha et al, 2026) at https://github.com/Katha-Sikdar/microservices-api_directory_psao-artifact/. It contains all measurements, the instrumentation, the analysis code and the scripts that generate every numeric value and every figure in this paper, including the C program and the script that builds each OpenSSL release, the exception baseline, and the cross-library harnesses in four

[languages with their pinned dependency manifests](#). Every number is generated from a file under the run or survey directories; none is entered by hand. The survey corpus is third-party source and is not redistributed; a manifest pinning every file examined by content hash stands in for it, covering 502 files across 457 repositories. Content hashes resolved for all of them; the repository HEAD commit resolved for 501 of 502, one repository having become unavailable between fetch and manifest. The content hash is the authoritative pin and is complete.

Competing interests. The authors declare no competing interests.

Funding. [The authors did not receive support from any organization for the submitted work.](#)

Author contributions. [Jannatul Ferdous Katha: conceptualisation, methodology, software, investigation, data curation, formal analysis, visualisation, writing – original draft, Tasmia Tahmid Prova: validation, writing – review and editing, Md Kishor Morol: supervision, writing – review and editing, Tze Hui Liew: supervision, project administration, writing – review and editing, Dip Nandi: supervision, writing – review and editing. All authors read and approved the final manuscript.](#)

Ethics approval. Not applicable. This study measures software behaviour and publicly available source code; it involves no human participants and no personal data.

References

- Acar Y, Backes M, Fahl S, et al (2017) Comparing the usability of cryptographic APIs. In: 2017 IEEE Symposium on Security and Privacy (SP), <https://doi.org/10.1109/SP.2017.52>
- Barrett E, Bolz-Tereick CF, Killick R, et al (2017) Virtual machine warmup blows hot and cold. *Proceedings of the ACM on Programming Languages* 1(OOPSLA):52:1–52:27. <https://doi.org/10.1145/3133876>
- Cabral B, Marques P (2007) Exception handling: A field study in Java and .NET. In: ECOOP 2007 – Object-Oriented Programming, *Lecture Notes in Computer Science*, vol 4609. Springer, pp 151–175, https://doi.org/10.1007/978-3-540-73589-2_8
- Chen J, Shang W (2017) An exploratory study of performance regression introducing code changes. In: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME), <https://doi.org/10.1109/ICSME.2017.13>
- Cito J, Schermann G, Wittern JE, et al (2017) An empirical analysis of the Docker container ecosystem on GitHub. In: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), <https://doi.org/10.1109/MSR.2017.67>
- Coarfa C, Druschel P, Wallach DS (2006) Performance analysis of TLS web servers. *ACM Transactions on Computer Systems* 24(1). <https://doi.org/10.1145/1124153.1124155>
- Costa D, Bezemer CP, Leitner P, et al (2021) What’s wrong with my benchmark results? Studying bad practices in JMH benchmarks. *IEEE Transactions on Software Engineering* 47(7):1452–1467. <https://doi.org/10.1109/TSE.2019.2925345>

- Curtsinger C, Berger ED (2013) STABILIZER: Statistically sound performance evaluation. In: Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13), pp 219–228, <https://doi.org/10.1145/2451116.2451141>
- Davis JC, Williamson ER, Lee D (2018) A sense of time for JavaScript and Node.js: First-class timeouts as a cure for event handler poisoning. In: Proceedings of the 27th USENIX Security Symposium, pp 343–359
- Detering D, Somorovsky J, Mainka C, et al (2017) On the (in-)security of JavaScript object signing and encryption. In: Proceedings of the 1st Reversing and Offensive-oriented Trends Symposium (ROOTS '17), <https://doi.org/10.1145/3150376.3150379>
- Egele M, Brumley D, Fratantonio Y, et al (2013) An empirical study of cryptographic misuse in Android applications. In: Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS '13), <https://doi.org/10.1145/2508859.2516693>
- Gaynor A, Kehrer P (2026) The state of OpenSSL for pyca/cryptography. pyca/cryptography project statement, <https://cryptography.io/en/latest/statements/state-of-openssl/>, maintainer statement, not peer reviewed; accessed 2026-09-24
- Georges A, Buytaert D, Eeckhout L (2007) Statistically rigorous Java performance evaluation. In: Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07), <https://doi.org/10.1145/1297027.1297033>
- Georgiev M, Iyengar S, Jana S, et al (2012) The most dangerous code in the world: Validating SSL certificates in non-browser software. In: Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS '12), <https://doi.org/10.1145/2382196.2382204>
- Gong L, Pradel M, Sen K (2015) JITProf: Pinpointing JIT-unfriendly JavaScript code. In: Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE '15), <https://doi.org/10.1145/2786805.2786831>
- Green M, Smith M (2016) Developers are not the enemy! the need for usable security APIs. IEEE Security & Privacy 14(5). <https://doi.org/10.1109/MSP.2016.111>
- Indela S, Kulkarni M, Nayak K, et al (2016) Helping Johnny encrypt: Toward semantic interfaces for cryptographic frameworks. In: Proceedings of the 2016 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2016), <https://doi.org/10.1145/2986012.2986024>
- Jin G, Song L, Shi X, et al (2012) Understanding and detecting real-world performance bugs. In: Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '12), pp 77–88, <https://doi.org/10.1145/2254064.2254075>

- Jones MB, Bradley J, Sakimura N (2015) JSON web token (JWT). Request for Comments RFC 7519, Internet Engineering Task Force, <https://doi.org/10.17487/RFC7519>
- Kalibera T, Jones R (2013) Rigorous benchmarking in reasonable time. In: Proceedings of the 2013 International Symposium on Memory Management (ISMM '13), <https://doi.org/10.1145/2464157.2464160>
- Kalliamvakou E, Gousios G, Blincoe K, et al (2014) The promises and perils of mining GitHub. In: Proceedings of the 11th Working Conference on Mining Software Repositories (MSR 2014), <https://doi.org/10.1145/2597073.2597074>
- Kaltenecker C, Mühlbauer S, Grebhahn A, et al (2023) Performance evolution of configurable software systems: An empirical study. *Empirical Software Engineering* 28(6). <https://doi.org/10.1007/s10664-023-10338-3>
- Katha JF, Prova TT, Morol MK, et al (2026) Replication package: where the cost of JWT verification actually goes. <https://github.com/Katha-Sikdar/microservices-api>, directory `psao-artifact/`
- Kery MB, Le Goues C, Myers BA (2016) Examining programmer practices for locally handling exceptions. In: Proceedings of the 13th International Conference on Mining Software Repositories (MSR '16), pp 484–487, <https://doi.org/10.1145/2901739.2903497>
- Krüger S, Späth J, Ali K, et al (2018) CrySL: An extensible approach to validating the correct usage of cryptographic APIs. In: 32nd European Conference on Object-Oriented Programming (ECOOP 2018), pp 10:1–10:27, <https://doi.org/10.4230/LIPIcs.ECOOP.2018.10>
- Laaber C, Leitner P (2018) An evaluation of open-source software microbenchmark suites for continuous performance assessment. In: Proceedings of the 15th International Conference on Mining Software Repositories (MSR '18), <https://doi.org/10.1145/3196398.3196407>
- Laaber C, Scheuner J, Leitner P (2019) Software microbenchmarking in the cloud. how bad is it really? *Empirical Software Engineering* 24(4). <https://doi.org/10.1007/s10664-019-09681-1>
- Lazar D, Chen H, Wang X, et al (2014) Why does cryptographic software fail? A case study and open problems. In: Proceedings of the 5th Asia-Pacific Workshop on Systems (APSys '14), <https://doi.org/10.1145/2637166.2637237>
- Leitner P, Cito J (2016) Patterns in the chaos—a study of performance variation and predictability in public IaaS clouds. *ACM Transactions on Internet Technology* 16(3). <https://doi.org/10.1145/2885497>
- Liu Y, Xu C, Cheung SC (2014) Characterizing and detecting performance bugs for smartphone applications. In: Proceedings of the 36th International Conference on Software Engineering (ICSE '14), pp 1013–1024, <https://doi.org/10.1145/2568225.2568229>

- Malka J, Zacchiroli S, Zimmermann T (2026) Docker does not guarantee reproducibility. arXiv preprint arXiv:2601.12811, no DOI registered; verified against the arXiv record
- Mytkowicz T, Diwan A, Hauswirth M, et al (2009) Producing wrong data without doing anything obviously wrong! In: Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09), <https://doi.org/10.1145/1508244.1508275>
- Mytkowicz T, Diwan A, Hauswirth M, et al (2010) Evaluating the accuracy of Java profilers. In: Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '10), pp 187–197, <https://doi.org/10.1145/1806596.1806618>
- Nadi S, Krüger S, Mezini M, et al (2016) Jumping through hoops. In: Proceedings of the 38th International Conference on Software Engineering (ICSE '16), <https://doi.org/10.1145/2884781.2884790>
- Naser A, Sadeghi Lahijani M, Wu C, et al (2020) Performance evaluation and modeling of cryptographic libraries for MPI communications. arXiv preprint arXiv:2010.06139, no DOI registered; verified against the arXiv record
- Nistor A, Jiang T, Tan L (2013a) Discovering, reporting, and fixing performance bugs. In: Proceedings of the 10th Working Conference on Mining Software Repositories (MSR '13), <https://doi.org/10.1109/MSR.2013.6624035>
- Nistor A, Song L, Marinov D, et al (2013b) Toddler: Detecting performance problems via similar memory-access patterns. In: Proceedings of the 35th International Conference on Software Engineering (ICSE '13), pp 562–571, <https://doi.org/10.1109/ICSE.2013.6606602>
- Ogasawara T, Komatsu H, Nakatani T (2006) EDO: Exception-directed optimization in Java. ACM Transactions on Programming Languages and Systems 28(1). <https://doi.org/10.1145/1111596.1111598>
- de Pádua GB, Shang W (2017) Studying the prevalence of exception handling anti-patterns. In: Proceedings of the 25th International Conference on Program Comprehension (ICPC '17), pp 328–331, <https://doi.org/10.1109/ICPC.2017.1>
- Parravicini A, Mueller T (2021) The cost of speculation: Revisiting overheads in the V8 JavaScript engine. In: 2021 IEEE International Symposium on Workload Characterization (IISWC), <https://doi.org/10.1109/IISWC53511.2021.00013>
- Rahaman S, Xiao Y, Afrose S, et al (2019) CryptoGuard: High precision detection of cryptographic vulnerabilities in massive-sized Java projects. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19), pp 2455–2472, <https://doi.org/10.1145/3319535.3345659>
- Renwick J, Spink T, Franke B (2019) Low-cost deterministic C++ exceptions for embedded systems. In: Proceedings of the 28th International Conference on Compiler Construction

- (CC 2019), <https://doi.org/10.1145/3302516.3307346>
- Selakovic M, Pradel M (2016) Performance issues and optimizations in JavaScript: An empirical study. In: Proceedings of the 38th International Conference on Software Engineering (ICSE '16), pp 61–72, <https://doi.org/10.1145/2884781.2884829>
- Selakovic M, Glaser T, Pradel M (2017) An actionable performance profiler for optimizing the order of evaluations. In: Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '17), <https://doi.org/10.1145/3092703.3092716>
- Shatnawi A, Al-Duwairi B, Smarneh A (2024) Comprehensive empirical study of Python JWT libraries. *Procedia Computer Science* 238:827–832. <https://doi.org/10.1016/j.procs.2024.06.099>
- Sheffer Y, Hardt D, Jones MB (2020) JSON web token best current practices. Request for Comments RFC 8725, Internet Engineering Task Force, <https://doi.org/10.17487/RFC8725>
- Sikeridis D, Kampanakis P, Devetsikiotis M (2020) Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH. In: Proceedings of the 16th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '20), <https://doi.org/10.1145/3386367.3431305>
- Song L, Lu S (2014) Statistical debugging for real-world performance problems. In: Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA '14), pp 561–578, <https://doi.org/10.1145/2660193.2660234>
- Staicu CA, Pradel M (2018) Freezing the web: A study of ReDoS vulnerabilities in JavaScript-based web servers. In: Proceedings of the 27th USENIX Security Symposium, pp 361–376
- Stefan P, Horký V, Bulej L, et al (2017) Unit testing performance in Java projects: Are we there yet? In: Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE '17), pp 401–412, <https://doi.org/10.1145/3030207.3030226>
- Traini L, Cortellesa V, Di Pompeo D, et al (2023) Towards effective assessment of steady state performance in Java software: Are we there yet? *Empirical Software Engineering* 28(1). <https://doi.org/10.1007/s10664-022-10247-x>
- Tuveri N, Brumley BB (2019) Start your ENGINes: Dynamically loadable contemporary crypto. In: 2019 IEEE Cybersecurity Development (SecDev), <https://doi.org/10.1109/SecDev.2019.00014>
- Weimer W, Necula GC (2004) Finding and preventing run-time error handling mistakes. In: Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '04), <https://doi.org/10.1145/1028976.1029011>
- Xu B, Jia S, Lin J, et al (2023) JWTKey: Automatic cryptographic vulnerability detection in JWT applications. In: Computer Security – ESORICS 2023, Lecture Notes in Computer Science,

vol 14346. Springer, https://doi.org/10.1007/978-3-031-51479-1_14

Yang J, Wang E, Chen J, et al (2026) Token time bomb: Evaluating JWT implementations for vulnerability discovery. In: Proceedings of the Network and Distributed System Security Symposium (NDSS '26)

Zaman S, Adams B, Hassan AE (2012) A qualitative study on performance bugs. In: Proceedings of the 9th IEEE Working Conference on Mining Software Repositories (MSR '12), pp 199–208, <https://doi.org/10.1109/MSR.2012.6224281>

Zerouali A, Mens T, Robles G, et al (2019) On the relation between outdated Docker containers, severity vulnerabilities, and bugs. In: Proceedings of the 26th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '19), pp 491–501, <https://doi.org/10.1109/SANER.2019.8668013>

Zhu X, She G, Xue B, et al (2023) Dissecting overheads of service mesh sidecars. In: Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC '23), <https://doi.org/10.1145/3620678.3624652>